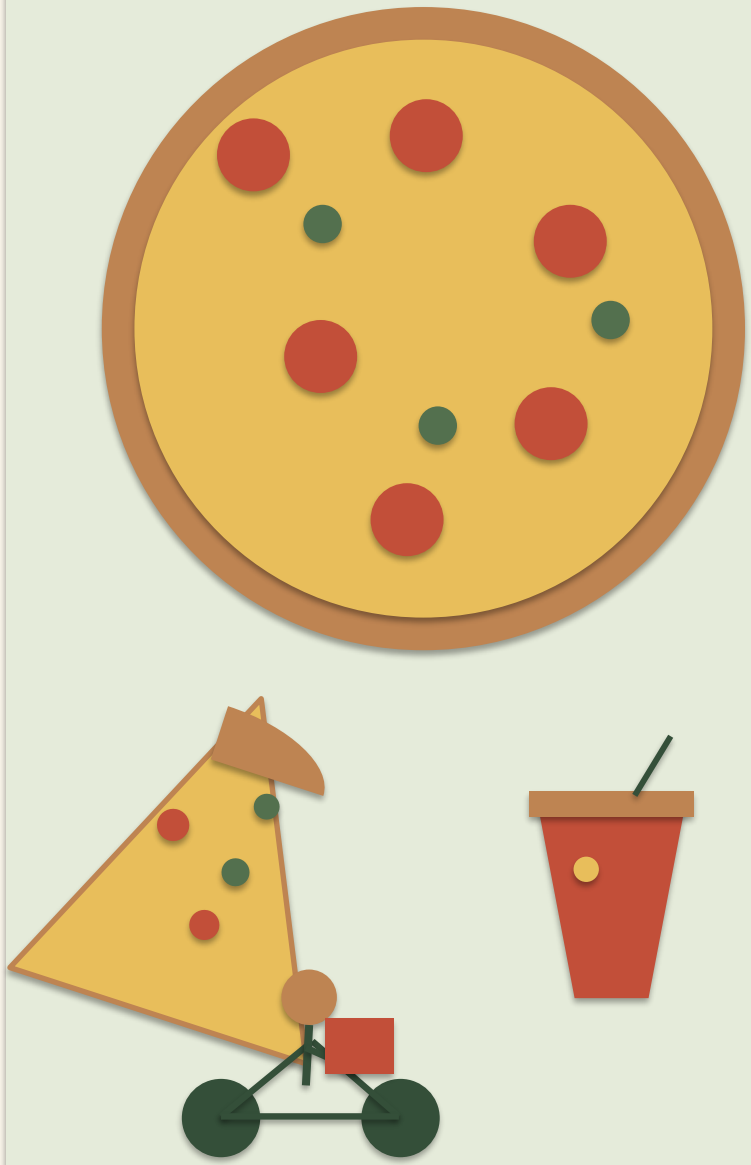
A DATA STORY, FRESH OUT OF THE OVEN

PIZZA RUNNER

SQL ANALYSIS

From orders to deliveries — exploring the story behind every slice.



PREPARED BY

Kunal Malviya

THE STORY BEHIND THE SLICE

A little intro

Pizza Runner began with a simple idea: bring the convenience of app-based delivery to a growing pizza business. Danny recruits runners, takes customer orders through an app, and needs reliable data to understand how the operation performs.

THE ANALYTICAL MISSION

01 Measure orders & pizza demand

02 Understand runner performance

03 Explore toppings & customisations

04 Evaluate revenue, ratings & costs

A practical SQL exploration of data cleaning, joins, aggregation and business insights.

THE DATA WE USED

- `runners`
- `customer_orders`
- `runner_orders`
- `pizza_names`
- `pizza_recipes`
- `pizza_toppings`



THE MENU

What’s on the table?

A

PIZZA METRICS

10 questions

Orders • customers • delivery counts • timing

B

RUNNER & CUSTOMER EXPERIENCE

7 questions

Sign-ups • prep time • distance • speed

C

INGREDIENT OPTIMISATION

6 questions

Recipes • extras • exclusions • ingredients

D

PRICING & RATINGS

5 questions

Revenue • extras • ratings • runner costs

How many pizzas were ordered?

```
SELECT COUNT(order_id) AS total_orders
FROM customer_orders;
```

Output

	total_orders
▶	14



How many unique customer orders were made?

```
SELECT COUNT(DISTINCT order_id)
AS total_unique_orders
FROM customer_orders;
```

Output

	total_unique_orders
▶	10



How many successful deliveries did each runner complete?

```sql
SELECT runner_id,
COUNT(order_id) AS order_count
FROM runner_orders
WHERE cancellation IS NULL
OR LOWER(TRIM(cancellation)) IN  ('','null')
GROUP BY 1;
```

Output

	runner_id	order_count
▶	1	4
	2	3
	3	1



How many of each pizza type were delivered?

```
SELECT pn.pizza_id, pn.pizza_name,
COUNT(co.pizza_id) AS deliver_count
FROM pizza_names pn
JOIN customer_orders co
ON pn.pizza_id = co.pizza_id
JOIN runner_orders ro
ON co.order_id = ro.order_id
WHERE ro.cancellation IS NULL
OR LOWER(TRIM(ro.cancellation)) IN ('','null')
GROUP BY pn.pizza_id, pn.pizza_name;
```

Output

	pizza_id	pizza_name	deliver_count
▶	1	Meatlovers	9
	2	Vegetarian	3



How many Vegetarian and Meat Lovers pizzas did each customer order?

```
SELECT co.customer_id, pn.pizza_name,
COUNT(pn.pizza_id) AS pizza_order_count
FROM customer_orders co
JOIN pizza_names pn
ON co.pizza_id = pn.pizza_id
GROUP BY 1,2
ORDER BY 1;
```

Output

customer_id	pizza_name	pizza_order_count
101	Meatlovers	2
101	Vegetarian	1
102	Meatlovers	2
102	Vegetarian	1
103	Meatlovers	3
103	Vegetarian	1
104	Meatlovers	3
105	Vegetarian	1



What was the maximum number of pizzas in one delivered order?

```
WITH pizza_delivered AS (SELECT ro.order_id,
COUNT(co.pizza_id) AS pizza_count
FROM runner_orders ro
JOIN customer_orders co
ON ro.order_id = co.order_id
WHERE ro.cancellation IS NULL
OR LOWER(TRIM(ro.cancellation)) IN ('','null')
GROUP BY ro.order_id)

SELECT MAX(pizza_count) AS max_pizza_delivered
FROM pizza_delivered;
```

Output

	max_pizza_delivered
▶	3



Per customer, how many delivered pizzas had changes vs. no changes?

```
SELECT co.customer_id, COUNT(co.pizza_id) AS total_delivered,
COUNT(CASE WHEN
(exclusions IS NOT NULL AND TRIM(exclusions) NOT IN ('','null'))
OR
(extras IS NOT NULL AND TRIM(extras) NOT IN ('','null'))
THEN co.pizza_id END) AS change_count,
COUNT(CASE WHEN
(exclusions IS NULL OR TRIM(exclusions) IN ('','null'))
AND
(extras IS NULL OR TRIM(extras) IN ('','null'))
THEN co.pizza_id END) AS no_change_count
FROM customer_orders co
JOIN runner_orders ro
ON co.order_id = ro.order_id
WHERE ro.cancellation IS NULL
OR LOWER(TRIM(ro.cancellation)) IN ('','null')
GROUP BY co.customer_id;
```

Output

customer_id	total_delivered	change_count	no_change_count
101	2	0	2
102	3	0	3
103	3	3	0
104	3	2	1
105	1	1	0



A08

How many delivered pizzas had both exclusions and extras?

```sql
SELECT COUNT(CASE WHEN
(co.exclusions IS NOT NULL AND TRIM(co.exclusions) NOT IN ('','null'))
AND
(co.extras IS NOT NULL AND TRIM(co.extras) NOT IN ('','null'))
THEN co.pizza_id END) AS total_pizza_delivered
FROM customer_orders co
JOIN runner_orders ro
ON co.order_id = ro.order_id
WHERE ro.cancellation IS NULL
OR LOWER(TRIM(ro.cancellation)) IN ('','null');
```

Output

	total_pizza_delivered
▶	1



A09

How many pizzas were ordered in each hour of the day?

```
SELECT HOUR(order_time) AS hour_of_day,
COUNT(pizza_id) AS pizza_ordered
FROM customer_orders
GROUP BY hour_of_day
ORDER BY hour_of_day;
```

Output

hour_of_day	pizza_ordered
11	1
13	3
18	3
19	1
21	3
23	3



How many orders were placed on each day of the week?

```sql
SELECT DAYOFWEEK(order_time) AS day_of_week,
COUNT(order_id) AS order_count
FROM customer_orders
GROUP BY 1
ORDER BY 1;

-- 2nd Appraoch
SELECT dayname(order_time) AS day_name,
COUNT(order_id) AS order_count
FROM customer_orders
GROUP BY DAYOFWEEK(order_time), DAYNAME(order_time)
ORDER BY DAYOFWEEK(order_time);
```

Output

day_name	order_count
Wednesday	5
Thursday	3
Friday	1
Saturday	5



How many runners signed up in each 1-week period?

```sql
WITH RECURSIVE bins AS(
SELECT DATE('2021-01-01') AS week_start
UNION ALL
SELECT DATE_ADD(week_start, INTERVAL 1 WEEK) AS n1
FROM bins WHERE week_start <
(SELECT MAX(DATE(registration_date)) FROM runners))

SELECT b.week_Start,
COUNT(r.runner_id) AS runner_count
FROM bins b JOIN runners r
ON DATE(r.registration_date) >= b.week_start
AND DATE(r.registration_date) < DATE_ADD(b.week_start, INTERVAL 1 WEEK)
GROUP BY b.week_start;
```

Output

week_Start	runner_count
2021-01-01	2
2021-01-08	1
2021-01-15	1



What was each runner's average time to reach HQ for pickup?

```
SELECT ro.runner_id,
ROUND(AVG(TIMESTAMPDIFF(MINUTE, co.order_time, ro.pickup_time)),2)
AS avg_time
FROM runner_orders ro
JOIN customer_orders co
ON ro.order_id = co.order_id
GROUP BY ro.runner_id;
```

Output

runner_id	avg_time
1	15.33
2	23.40
3	10.00



B03

Is pizza quantity related to order preparation time?

```sql
SELECT co.order_id,
COUNT(co.order_id) AS pizza_count,
ROUND(MAX(TIMESTAMPDIFF(MINUTE, co.order_time, ro.pickup_time)),2)
AS time_taken
FROM customer_orders co
JOIN runner_orders ro
ON co.order_id = ro.order_id
WHERE ro.cancellation IS NULL
OR LOWER(TRIM(ro.cancellation)) IN ('','null')
GROUP BY order_id;
```

Output

order_id	pizza_count	time_taken
1	1	10
2	1	10
3	2	21
4	3	29
5	1	10
7	1	10
8	1	20
10	2	15



B04

What was the average delivery distance for each customer?

```
SELECT co.customer_id,
ROUND(AVG(CAST(REPLACE(ro.distance, 'km','') AS DECIMAL(10,2))),2)
AS distance_travelled
FROM customer_orders co
JOIN runner_orders ro
ON co.order_id = ro.order_id
GROUP BY co.customer_id;
```

Output

customer_id	distance_travelled
101	13.33
102	16.73
103	17.55
104	10.00
105	25.00



B05

What was the gap between the longest and shortest delivery times?

```
SELECT
MIN(CAST(REGEXP_REPLACE(duration, '[^0-9]','') AS UNSIGNED))
AS minimum_delivery_time,
MAX(CAST(REGEXP_REPLACE(duration, '[^0-9]','') AS UNSIGNED))
AS maximum_delivery_time,
(MAX(CAST(REGEXP_REPLACE(duration, '[^0-9]','') AS UNSIGNED)) -
MIN(CAST(REGEXP_REPLACE(duration, '[^0-9]','') AS UNSIGNED)))
AS difference
FROM runner_orders
WHERE cancellation IS NULL
OR LOWER(TRIM(cancellation)) IN ('','null');
```

Output

	minimum_delivery_time	maximum_delivery_time	difference
▶	10	40	30



B06

What was each runner's average speed per delivery, and was there a trend?

```sql
SELECT runner_id,
ROUND(CAST(REPLACE(distance, 'km','') AS DECIMAL(10,2)) /
CAST(REGEXP_REPLACE(duration, '[^0-9]','') AS UNSIGNED),2)
AS speed
FROM runner_orders
WHERE cancellation IS NULL
OR LOWER(TRIM(cancellation)) IN ('','null');
```

Output

runner_id	speed
1	0.63
1	0.74
1	0.67
2	0.59
3	0.67
2	1.00
2	1.56
1	1.00



What was the successful delivery percentage for each runner?

```
WITH total_delivery AS(
SELECT runner_id,
COUNT(runner_id) AS assigned_delivery
FROM runner_orders
GROUP BY runner_id
),

successful_delivery AS(
SELECT runner_id,
COUNT(runner_id) AS delivered
FROM runner_orders
WHERE cancellation IS NULL
OR LOWER(TRIM(cancellation)) IN ('','null')
GROUP BY runner_id
)

SELECT td.runner_id, td.assigned_delivery,
sd.delivered,
((sd.delivered*100) / (td.assigned_delivery))
AS percentage
FROM total_delivery AS td
JOIN successful_delivery AS sd
ON td.runner_id = sd.runner_id;
```

Output

runner_id	assigned_delivery	delivered	percentage
1	4	4	100.0000
2	4	3	75.0000
3	2	1	50.0000



C01

What are the standard ingredients in each pizza?

```
SELECT pn.pizza_id, pn.pizza_name,
pr.toppings
FROM pizza_names pn
JOIN pizza_recipes pr
ON pn.pizza_id = pr.pizza_id;
```

Output

pizza_id	pizza_name	toppings
1	Meatlovers	1, 2, 3, 4, 5, 6, 8, 10
2	Vegetarian	4, 6, 7, 9, 11, 12



What was the most commonly added extra?

```
SELECT pt.topping_id, pt.topping_name,
COUNT(*) AS extra_count
FROM pizza_toppings pt
JOIN customer_orders co
ON FIND_IN_SET(pt.topping_id, REPLACE(co.extras, ' ','')) > 0
WHERE co.extras IS NOT NULL
AND LOWER(TRIM(co.extras)) NOT IN ('','null')
GROUP BY 1,2;
```

Output

topping_id	topping_name	extra_count
1	Bacon	4
5	Chicken	1
4	Cheese	1



C03

What was the most common exclusion?

```
SELECT pt.topping_id, pt.topping_name,
COUNT(*) AS exclusion_count
FROM pizza_toppings pt
JOIN customer_orders co
ON FIND_IN_SET(pt.topping_id, REPLACE(co.exclusions,' ','')) > 0
WHERE co.exclusions IS NOT NULL
AND LOWER(TRIM(co.exclusions)) NOT IN ('','null')
GROUP BY 1,2;
```

Output

topping_id	topping_name	exclusion_count
4	Cheese	4
6	Mushrooms	1
2	BBQ Sauce	1



C06

What was the total usage of each ingredient across delivered pizzas?

```sql
SELECT pt.topping_id, pt.topping_name,
COUNT(*) AS quantity
FROM pizza_toppings pt
JOIN pizza_recipes pr
ON FIND_IN_SET(pt.topping_id, REPLACE(pr.toppings,' ','')) >0
JOIN customer_orders co
ON co.pizza_id = pr.pizza_id
JOIN runner_orders ro
ON ro.order_id = co.order_id
WHERE ro.cancellation IS NULL
OR LOWER(TRIM(ro.cancellation)) IN ('','null')
GROUP BY 1,2
ORDER BY quantity DESC;
```

Output

topping_id	topping_name	quantity
4	Cheese	12
6	Mushrooms	12
1	Bacon	9
2	BBQ Sauce	9
3	Beef	9
5	Chicken	9
8	Pepperoni	9
10	Salami	9
7	Onions	3
9	Peppers	3
11	Tomatoes	3
12	Tomato Sauce	3



D01

How much revenue was earned with fixed pizza prices and no delivery fees?

```sql
WITH costing AS (SELECT co.pizza_id, pn.pizza_name,
(CASE WHEN co.pizza_id = 1 THEN 12 ELSE 10 END) AS cost
FROM pizza_names pn
JOIN customer_orders co
ON pn.pizza_id = co.pizza_id
JOIN runner_orders ro
ON co.order_id = ro.order_id
WHERE ro.cancellation IS NULL
OR LOWER(TRIM(ro.cancellation)) IN ('','null'))

SELECT SUM(cost) AS revenue
FROM costing;
```

Output

	revenue
▶	138



D02

How would revenue change with a $1 charge for each extra?

```sql
WITH costing AS (SELECT co.pizza_id, pn.pizza_name,
(CASE WHEN co.pizza_id = 1 THEN 12 ELSE 10 END) AS pizza_cost,
(CASE WHEN co.extras IS NOT NULL
AND LOWER(TRIM(co.extras)) NOT IN ('','null') THEN 1 ELSE 0 END)
AS extra_charge
FROM pizza_names pn
JOIN customer_orders co
ON pn.pizza_id = co.pizza_id
JOIN runner_orders ro
ON co.order_id = ro.order_id
WHERE ro.cancellation IS NULL
OR LOWER(TRIM(ro.cancellation)) IN ('','null'))

SELECT SUM(pizza_cost + extra_charge) AS revenue
FROM costing;
```

Output

	revenue
▶	141



D04

Join successful deliveries with customer, runner, rating and delivery metrics.

```
SELECT co.customer_id, co.order_id,
ro.runner_id, co.order_time, ro.pickup_time,
(TIMESTAMPDIFF(MINUTE, co.order_time, ro.pickup_time))
AS time_between_order_and_pickup,
CAST(REGEXP_REPLACE(ro.duration, '[^0-9]','') AS UNSIGNED)
AS duration,
ROUND(CAST(REPLACE(ro.distance, 'km','') AS DECIMAL(10,2)) /
CAST(REGEXP_REPLACE(ro.duration, '[^0-9]','') AS UNSIGNED),2)
AS speed,
COUNT(co.pizza_id) AS total_pizza
FROM customer_orders co
JOIN runner_orders ro
ON co.order_id = ro.order_id
WHERE ro.cancellation IS NULL
OR LOWER(TRIM(ro.cancellation)) IN ('','null')
GROUP BY co.customer_id, co.order_id, ro.runner_id,
co.order_time, ro.pickup_time, duration, speed;
```

Output

customer_id	order_id	runner_id	order_time	pickup_time	time_between_order_and_pickup	duration	speed	total_pizza
101	1	1	2020-01-01 18:05:02	2020-01-01 18:15:34	10	32	0.63	1
101	2	1	2020-01-01 19:00:52	2020-01-01 19:10:54	10	27	0.74	1
102	3	1	2020-01-02 23:51:23	2020-01-03 00:12:37	21	20	0.67	2
103	4	2	2020-01-04 13:23:46	2020-01-04 13:53:03	29	40	0.59	3
104	5	3	2020-01-08 21:00:29	2020-01-08 21:10:57	10	15	0.67	1
105	7	2	2020-01-08 21:20:29	2020-01-08 21:30:45	10	25	1.00	1
102	8	2	2020-01-09 23:54:33	2020-01-10 00:15:02	20	15	1.56	1
104	10	1	2020-01-11 18:34:49	2020-01-11 18:50:20	15	10	1.00	2



D05

How much remained after paying runners $0.30 per kilometre?

```sql
WITH pizza_costing AS (SELECT co.order_id,
SUM(CASE WHEN co.pizza_id = 1 THEN 12
      WHEN co.pizza_id = 2 THEN 10 END) AS cost
FROM customer_orders co
JOIN runner_orders ro
ON co.order_id = ro.order_id
WHERE ro.cancellation IS NULL
OR LOWER(TRIM(ro.cancellation)) IN ('','null')
GROUP BY 1),

runner_money AS (SELECT order_id, runner_id,
ROUND(CAST(REPLACE(distance, 'km', '') AS DECIMAL(10,2)) * 0.30, 2)
AS runner_payment
FROM runner_orders
WHERE cancellation IS NULL
OR LOWER(TRIM(cancellation)) IN ('','null'))

SELECT SUM(pc.cost) - SUM(rm.runner_payment)
AS money_left
FROM pizza_costing pc
JOIN runner_money rm
ON pc.order_id = rm.order_id;
```

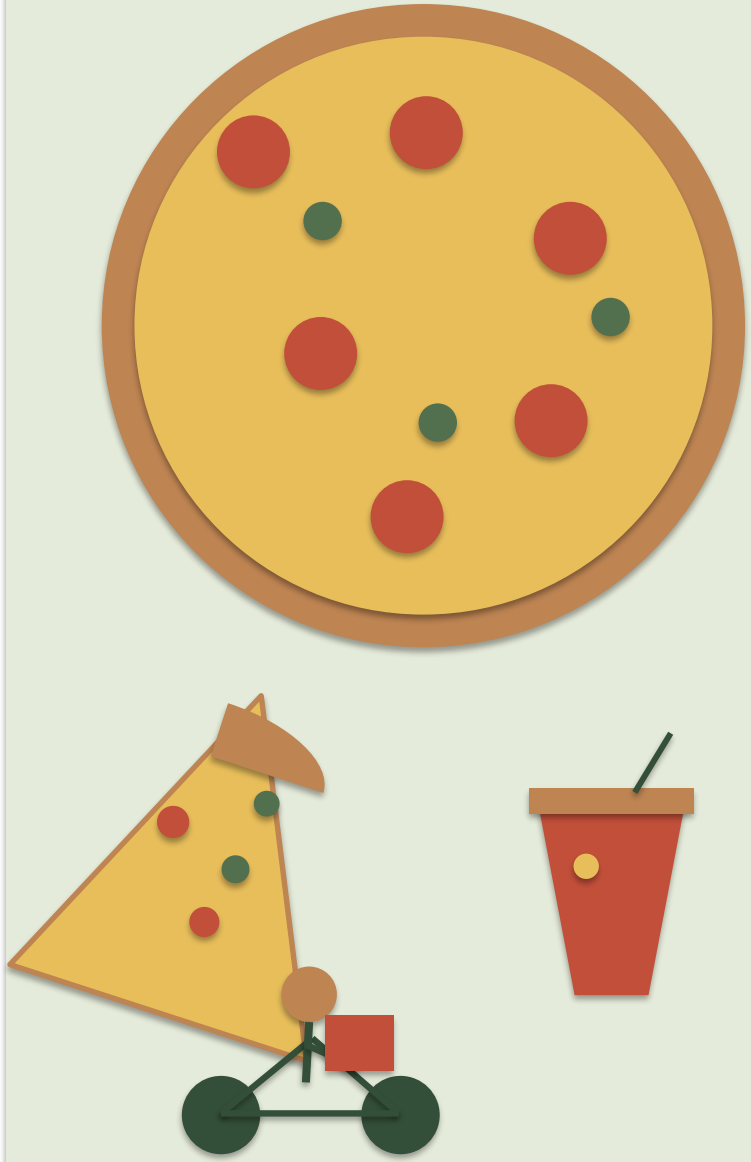
Output

	money_left
▶	94.44



THANK YOU

"Good data is like a great pizza — the right ingredients, perfectly combined, turn raw numbers into something everyone loves."



PREPARED BY

Kunal Malviya